



ENSAM Casablanca

Filière: Intelligence Artificielle et Génie Informatique (IAGI 2)

Multi-Agent Deep Reinforcement Learning for Cooperative Soccer: Continuous Physics-Based Control in MuJoCo Simulator

IMAD EDDINE OUKRATI & ECHCHYOUGH I MOHAMED

Supervisor: Prof. Hirchoua Badr

A report submitted in partial fulfilment of the requirements of
ENSAM Casablanca
Module: Deep Reinforcement Learning

June 7, 2026

Declaration

I, IMAD EDDINE OUKRATI and ECHCHYOUGH I MOHAMED, of the Intelligence Artificielle et Génie Informatique (IAGI 2) programme, ENSAM Casablanca, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

IMAD EDDINE OUKRATI & ECHCHYOUGH I MOHAMED
June 7, 2026

Abstract

We train/assess MADRL players on a 2vs2 competitive football setting in a MuJoCo physics environment utilizing PettingZoo interface to allow fully autonomous players to learn how to chase a ball, play defence, and work together to score a goal from raw input data and no manual rules. Our primary methodology uses PPO reinforcement learning with a shared agent parameterization, enabling N/A solution for continuous action/state spaces, and N/A solution for extremely sparse reward issues were addressed by implementing/using custom reward shaping wrapper that calculates instantaneous rewards based on physical metrics such as distance penalties for chasing the ball, and speed of players toward goal.

In addition, we evaluate tactical play via spatial analysis software which produces 2D pitch coverage heatmaps, team spatial distribution (spread), and team possession statistics. Training occurs via parallelized vectorized training runs on a single Google Colab T4 GPU. The resulting PPO agents exhibit excellent ball interception skills and team coordination/individual coordination. Additionally, the hexbin maps of the pitches indicate that teams divide the pitch naturally into offensive/defensive zones (by colour) in an approximate 50:50 possession distribution with 48% (red) and 51% (blue). Continuous reward wrappers and agent parameter sharing produce agents that have learned to overcome physics-based non-stationarity.

Keywords: Multi-Agent Reinforcement Learning, Proximal Policy Optimisation, Parameter Sharing, Reward Shaping, MuJoCo Physics, Soccer Analytics, Spatial Heatmaps, PettingZoo.

Acknowledgements

We would like to express our deepest appreciation to Prof. Hirchoua Badr for providing such excellence in his teaching of the entire deep reinforcement learning course at ENSAM Casablanca. The thoroughness of the course material, the lab sessions, and the expert instruction in understanding both the theoretical foundations of reinforcement learning and how to implement them were essential to our completion of this project.

We also want to thank the entire open-source community who developed and contributed to the two libraries that made this project possible: the DeepMind Control Suite (and corresponding MuJoCo Physics Engine), the PettingZoo and Shimmy multi-agent frameworks, Stable-Baselines3, and the Open AI Gym library. The availability of their library and documentation as open-source is what allowed us to execute this project.

Lastly, thank you to Google for providing us access to GPU resources via Google Co-Laboratory that allowed us to train our Deep Reinforcement Learning Agents in the short time frame of this project.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	2
1.1 Project Presentation	2
1.1.1 Background	2
1.1.2 Problem Statement	2
1.1.3 Aims and Objectives	3
1.1.4 Solution Approach	3
1.2 Project Management	4
1.3 Summary	4
2 Literature Review	5
2.1 Reinforcement Learning Foundations	5
2.1.1 Markov Decision Process	6
2.1.2 Return and Value Functions	6
2.1.3 The Bellman Equation	7
2.2 Value-Based Methods	7
2.2.1 Q-Learning	7
2.2.2 Deep Q-Network (DQN)	8
2.3 Policy-Based and Actor-Critic Methods	8
2.3.1 Policy Gradient Theorem and REINFORCE	8
2.3.2 Proximal Policy Optimisation (PPO)	9
2.4 Multi-Agent Reinforcement Learning	10
2.4.1 Parameter Sharing	10
2.4.2 Centralised Training, Decentralised Execution (CTDE)	10
2.5 Critique of the Review	10
2.6 Summary	11
3 Methodology	12
3.1 Multi-Agent System Architecture	12
3.2 MuJoCo Soccer 2v2 Simulation Environment	13
3.2.1 Physical and Kinematic Specifications	13
3.2.2 State Observation and Action Spaces	13
3.3 Proximal Policy Optimisation (PPO) with Shared Policies	14
3.3.1 Custom Reward Shaping	14
3.3.2 Parameter Sharing and Environment Wrapping	15
3.3.3 PPO Hyperparameters	16

3.3.4	Training Infrastructure	16
3.4	Ethical Considerations	16
3.5	Summary	17
4	Results	18
4.1	Impact of Custom Reward Shaping (Objectives O1–O2)	18
4.2	Live Match Evaluation Metrics (Objective O3)	19
4.3	Spatial Pitch Coverage Density Heatmaps (Objective O4)	20
4.4	Summary of Results	21
5	Discussion and Analysis	22
5.1	Interpretation of Results	22
5.1.1	Cooperative Multi-Agent Coordination via PPO	22
5.1.2	Effectiveness of Custom Reward Shaping	22
5.1.3	Spatial Coverage Heatmaps and Teammate Spacing Dynamics	23
5.2	Significance of the Findings	23
5.3	Limitations	23
5.4	Summary	24
6	Conclusions and Future Work	25
6.1	Conclusions	25
6.2	Future Work	26
6.3	Summary	27
7	Reflection	28
7.1	Summary	29
	References	30
	Appendices	32
A	Project Source Code Overview	32

List of Figures

3.1	System architecture of the multi-agent reinforcement learning soccer simulator pipeline.	13
3.2	Visual representation of the 3D soccer pitch coordinate axes and egocentric agent observations.	14
4.1	PPO Soccer 2v2 evaluation results over 10 matches. (Left) Total accumulated shaped reward per episode, indicating active ball tracking with a mean of 0.73. (Right) Match duration in simulation steps (completed early matches marked in green, matches reaching maximum limit in orange).	19
4.2	Multi-Agent RL Soccer Spatial Coverage Heatmap. (Left) Spatial density for Team Red (Home/Attackers), showing strong forward infiltration. (Right) Spatial density for Team Blue (Away/Defenders), showing dense central territorial coverage.	20

List of Tables

1.1	Project management timeline	4
3.1	PPO hyperparameters for MuJoCo Soccer 2v2.	16
4.1	Summary of results mapped to project objectives.	21
A.1	Main Python dependencies used in this project.	32

List of Abbreviations

RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimisation
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
TRPO	Trust Region Policy Optimisation
MAPPO	Multi-Agent Proximal Policy Optimisation
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
CTDE	Centralised Training, Decentralised Execution
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MLP	Multi-Layer Perceptron
TD	Temporal Difference
MSE	Mean Squared Error
ENSAM	École Nationale Supérieure des Arts et Métiers
IAGI	Ingénierie Avancée en Génie Informatique
API	Application Programming Interface

General Introduction

Multi-agent reinforcement learning (MARL) is the application of reinforcement learning (RL) to multiple agent systems where the agents utilize their previous experiences to improve their performance through trial-and-error learning. Unlike traditional supervised learning approaches where learning occurs using an existing dataset, MARL allows agents to try out different possible actions and find alternatives to control and navigate in the environment without needing to have specific programming instructions to control actions in certain ways.

This research explores the application of MARL in developing a RL-based 3D physical soccer environment based on the MuJoCo physics engine. The 3D soccer simulation enables two teams of four players (2 vs 2) to play against each other without any set rules written into the code base of the soccer simulation application. The agents or players are required to learn how to control their bodies and coordinate their efforts in order to run to the ball, make passes to teammates, and kick the ball into the goal to score points. The task becomes progressively more complicated by using continuous action spaces, the interacting dynamic relationships between agents, as well as the rapidly changing environment due to the real-time actions of all five agents updating policy simultaneously.

In this project, we will focus our study on the theoretical underpinnings of modern reinforcement learning techniques by initially studying issues associated with discrete variables (tabular Q learning and discrete Q networks).

Organization of the report

The remainder of this report will be organized as follows: Chapter 1 describes the context of our host organization (ENSAM Casablanca), project scope and specific objectives; Chapter 2 examines the mathematical basis for Reinforcement Learning, including Markov Decision Processes (MDPs), value based methods, and Continuous Actor-Critic algorithms; Chapter 3 covers our methodology in detail, including environment setup, custom reward wrappers, and hyperparameter settings; Chapter 4 provides our quantitative evaluation of the research data collected, with examples of both rewards and 2-dimensional spatial hexbin heat maps; Chapter 5 discusses cooperative behaviours among teams, team spreads, and physical limitations associated with cooperation; Chapter 6 wraps up this research project and provides thoughts on future directions for the project; and Chapter 7 provides an individual engineering and learning reflection on the work done.

Chapter 1

Introduction

This chapter will explain the context of our project, the multi-agent challenges that we are tackling, the engineering objectives we will pursue, the methodology and solution approaches we will use, and the project management timeline for the project.

1.1 Project Presentation

1.1.1 Background

The concept of Deep Reinforcement Learning (DRL) has brought a new level of innovation to autonomous agent development. Instead of creating manual rules for every situation, we allow the agent to learn through its own experiences in a given environment. Rather than relying on a programmer to predict every single possible scenario and create an appropriate rule for each scenario, the agent develops effective strategies through experimentation with the help of a reward signal that reflects the quality of the decisions made. Successful examples of DRL include AlphaGo ([Silver et al., 2016](#)) and deep Q learning networks which played games like Atari ([Mnih et al., 2015](#)) where both used reinforcement learning to achieve strategic intuition and behavioral complexity that could never be produced by a human programmer through hard coding. These developments led to a great deal of interest in extending DRL to even more challenging domains such as robot locomotion and real-time strategy gaming.

MARL takes this idea even further than DRL by introducing multiple agents into one environment learning simultaneously. This reality adds an entirely new level of challenge to learning; now not only must an agent learn how to work by itself, but also learn how to change its behavior based on the actions of the other agents in the environment. Depending on the type of tasks being tested, it is possible for a number of different types of structures to exist such as;

1.1.2 Problem Statement

Our main goal is to explore how we can use only reinforcement learning in training multiple autonomous agents to work, cooperate and compete in a physics-based environment that has continuous action space. There are four specific behaviors that four physically based agents need to learn from nothing:

1. The ability to control their joints and travel around in the continuous three dimensional pitch.
2. Locate, chase and intercept a moving soccer ball.

3. Coordinate with a teammate to carry the ball toward the goal.
4. Defend their own goal against two other opponents and attempt to score against them.

Due to this nature of the task, it is very difficult to accomplish. Goals will not happen often when executing random policies, resulting in a sparse reward signal. Moreover, state space contains continuous position, direction and velocity, and partial-timed learning will violate the Markov property for each individual agent.

1.1.3 Aims and Objectives

Our goal is to investigate the design, training, and evaluation of a cooperative 2-on-2 (2v2) soccer team using a continuous three-dimensional (3D) MuJoCo simulation with different analysis methods and train the agents using Proximal Policy Optimization (PPO), reward shaping wrappers, and spatial analysis.

Objectives:

1. Create and set up the MuJoCo 2v2 soccer environment by utilizing the PettingZoo and Shimmy wrappers so that each player in the 4-player simulation can interact in a stable manner across all four players in the physical environment.
2. Design a custom reward shaping wrapper to change goal-based (i.e., sparse) rewards to be dense (i.e., immediate) based on the distance and/or velocity of the agent to the goal so that an agent can receive useful and meaningful feedback during an episode, instead of just when the agent scores.
3. Use PPO along with a shared policy network and/or parameter sharing across all agents in order to train them to be able to exhibit cooperative behavior consistently and reduce sample complexity to improve the stability of the training in a non-stationary multi-agent environment.
4. Create a soccer analytics pipeline to analyze soccer data by extracting and visualizing the following metrics: team possession %, spatial spread index, and hexbin density heatmap, to provide a quantitative basis for evaluating path developed tactical organization and positional role separation among agents.

1.1.4 Solution Approach

Four major components make up our engineering pipeline:

- **Stage 1 — Environment Integration:** To create a stable and fast parallel vectorized interface with multiple instances of the MuJoCo (2v2) soccer environment, we use PettingZoo and Shimmy compatibility wrappers. This not only gives us a single interface to all environments, but it also allows us to collect experience batches as quickly and efficiently as possible.
- **Stage 2 — Reward Wrapper Logic:** We create a new class (`SoccerRewardShapingWrapper`) to enhance the sparse goal-based reward signal with dense, physics-based feedback about how far away an agent is from the ball and how fast it is moving toward the goal, making sure that all agents receive meaningful feedback every step of the way.

- **Stage 3 — Policy Optimization:** We use a single shared PPO (actor-critic) network for all teammates and use Google Colab to train using 1 T4 GPU and multiprocessing to speed up the collection of experiences and updates to the policy across all the environments.
- **Stage 4 — Spatial and Game Analysis:** After training is finished, we run dedicated post-training evaluation episodes to log the position of the agent during each game. This positional data is then processed to create 2D hexbin density heatmaps showing coverage of the pitch and possession rate calculations for each team to quantify their ball control relatively.

1.2 Project Management

We scheduled our project over a three-week period. Table 1.1 outlines the key phases and timelines of our work.

Table 1.1: Project management timeline

Phase	Tasks	Duration
1. Setup	Dependency installation, MuJoCo configuration, API wrapper checks	Week 1
2. Modeling	Reward shaping design, state-action interface analysis	Week 1–2
3. Implementation	Environment wrappers, training scripts, Colab notebook structure	Week 2
4. Training	PPO model training on Colab, checkpoint saving	Week 2–3
5. Analysis	Team possession metrics, spatial coverage heatmaps	Week 3
6. Compilation	Writing the LaTeX report and finalizing code comments	Week 3

1.3 Summary

This chapter describes the framework, problem statement and goals for our multi-agent soccer project. We established four clearly defined objectives: integrating into an environment, shaping the reward function, optimising the policy, and performing spatial analysis. We also produced a complete solution workflow and timeline. These objectives demonstrated the complexity of the project due to the effects of continuous action spaces, non-stationary multi-agents, and sparse reward signals all typical of environments with goal scoring. In Chapter 2, we will provide theoretical basis and literature review associated with reinforcement learning, such as Markov Decision Processes, and value-based methods of reinforcement learning; as well as algorithms of continuous actor-critic methods that serve as a foundation for our PPO based training.

Chapter 2

Literature Review

The main purpose of this chapter is to provide a conceptual basis for our implementation based on the class materials of Prof. Hirchoua Badr at ENSAM Casablanca. The foundation of our project will be placed into the context of the significant accomplishments in reinforcement learning that have taken place recently, and discuss the challenges that are present when moving from discrete to continuous environments, and provide justifications for the use of certain algorithms. In order to accomplish this, we utilize the Markov Decision Process (MDP) framework to provide a foundational definition around which all reinforcement learning will be built. After defining and demonstrating MDPs, we will discuss and compare the two most popular discrete state action methods that are used within the MDP framework (Tabular Q-Learning and Deep Q-Networks) and show their limitations when applied to high dimensional continuous control problems. After comparing and contrasting these two discrete state action methods, we will move forward with a discussion regarding the introduction of policy gradient and actor-critic methods, and explain the rationale behind selecting Proximal Policy Optimization (PPO) as the optimum algorithm for our environment, based on the stability guarantees it offers and the applicability of continuous action spaces within our problem domain. Ultimately, we hope to position our work among the larger objectives of Multi-Agent Reinforcement Learning (MARL) and to summarize the contributions that have had a significant impact on both cooperative and competitive multi-agent reinforcement learning, and highlight the gaps in the theoretical framework related to multi-agent reinforcement learning that have yet to be addressed.

2.1 Reinforcement Learning Foundations

Reinforcement Learning is a set of mathematical methods that helps to train “agents” or “decision makers” to make a series of choices (or decisions) in an unpredictable environment (Sutton and Barto, 2018). The fundamental way that learning occurs through reinforcement learning is via trial-and-error. The agent makes an observation at some point in a state, uses its policy to select a choice according to that observation, receives a reward for its action, and then moves on to another state. The primary aim of the agent will be to obtain an accurate policy to maximize the expected cumulative returned outcome of the decisions made so far; i.e., maximize the average discounted reward, which is determined by a “discount factor” between the future and present rewards. Unlike supervised learning techniques, the agent does not have access to any pre-defined labels, but rather can only use the reward signal to assess the quality of its previous actions by “exploring” its environment. Thus, reinforcement learning techniques are particularly useful for solving sequential decision problems, where there may not be a simple way of describing how to act in each specific situation; however, the

agent can learn how to behave correctly by interacting with the environment and observing the rewards that it receives as a result of its actions.

2.1.1 Markov Decision Process

An MDP can be used to model the interactions between an agent and its environment. An MDP consists of five components which can be represented as a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The first component $\mathcal{S}$ is the set of all possible states that the agent can find itself in when interacting with the environment, as the second component $\mathcal{A}$ consists of the action space containing all the actions available to the agent at state $s \in \mathcal{S}$ while interacting with the environment. The third component $P(s'|s, a)$ defines the state transition probability function representing the probabilities associated with the agent transitioning from state s to another state s' , based upon the agent taking action $a \in \mathcal{A}$. The fourth component $R(s, a)$ represents the immediate payoff, or reward, that the agent receives as a result of taking a specific action a while in state s . The final component $\gamma \in [0, 1)$ represents the value that the agent places upon receiving future rewards through time versus receiving immediate rewards; the discount factor γ is zero or one, where zero represents full value placed on future rewards and one represents total value placed on current rewards. The Markov assumption states that the state at time t , denoted s_t , and the agent's action at time t , denoted a_t , will define the future state s_{t+1} and subsequently will define every past state.

$$\mathbb{P}[S_{t+1} | S_t] = \mathbb{P}[S_{t+1} | S_1, \dots, S_t]. \quad (2.1)$$

2.1.2 Return and Value Functions

The agent's goal is to maximize the expected discounted return, which represents the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (2.2)$$

To make predictions about future rewards based on a policy, we need to create two types of value functions. The State Value function $V^\pi(s)$ values the total return (the sum of future discounted values) if an agent were to follow a policy π from the current state s only. It tells us whether it is good for the agent to be in that state by measuring the long-term reward for an agent occupying a specific state s . It represents an agent's long-term expected total reward by taking into consideration every expected reward the agent will receive by continuing to follow its policy π from this state onward:

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s] \quad (2.3)$$

The Action Value function $Q^\pi(s, a)$ (also known as a Q-function) calculates the total return (sum of expected discounted returns) of taking a specific action a in the current state s and then following the same policy π until the end of time. It tells us how good it is for an agent to perform an action a (by choosing an action out of all possible actions $\mathcal{A}$) from a specific state s :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a] \quad (2.4)$$

The Action Value function $Q^\pi(s, a)$ measures an action a in conjunction with the state s , whereas the State Value function $V^\pi(s)$ only measures the state s . So, the primary difference between the two value functions is what they are measuring: the State Value function $V^\pi(s)$ defines the quality of the state s itself, while the Action Value function $Q^\pi(s, a)$ defines the quality of a state-action pair (s, a) .

2.1.3 The Bellman Equation

The Bellman equation breaks down the value functions recursively. For the optimal state-value function $V^*(s)$, we write (Bellman, 1957):

$$V^*(s) = \max_a \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s'|s, a) V^*(s') \right]. \quad (2.5)$$

The recursive format of the expression above gives rise to two important areas of research; dynamic programming and temporal-difference learning. The purpose of these two areas of research is to enable an agent to calculate value functions without having knowledge of the entire future path. By definition, the value of being in a state is comprised of the maximum value an agent can immediately receive as a reward upon taking action, plus the value of the subsequent state at the time the action was taken (multiplied by some discount/cost factor). The agent always chooses the action that provides the highest long-run expected value, by choosing the maximum expected return for each action from its current state. By taking the maximum value of a particular action, it simplifies the problem of calculating long-term expected values to a one step lookahead, allowing for computation. Additionally, it does not require that agents have the entire set of transition probabilities $P(s'|s, a)$ pre-calculated; therefore, an agent is able to use an iterative process of learning through repetition (through interaction with the environment) to estimate $V^*(s)$, which is the basis of temporal-difference learning methods, such as Q-Learning.

2.2 Value-Based Methods

Value-based algorithms learn how to calculate an estimate for the action-value function $Q(s, a)$. By choosing actions that provide the maximum estimated value in terms of this function, these algorithms indirectly create policies and pick the action with the highest estimated value in every state, to be considered a valid greedy action. This means that the quality of the policy created through value-based methods is dependent on how well the algorithm has learned the correct estimate of $Q(s, a)$.

Thus, the closer the agent learns to $Q^*(s, a)$, the closer its behaviour will be to an optimal policy π^* . It is important to note that value-based techniques work well for discrete action spaces since the cost of evaluating and comparing all available actions $a \in \mathcal{A}$ at each timestep t is manageable by an agent.

2.2.1 Q-Learning

Q-Learning (Watkins and Dayan, 1992) is a classic off-policy temporal-difference algorithm. It updates its value estimates using the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (2.6)$$

where α represents the learning rate. To balance exploration and exploitation, the agent uses an ε -greedy policy. With probability ε it chooses an action at random, and with probability $1 - \varepsilon$ it exploits its current knowledge. Q-learning is guaranteed to converge to the optimal policy, but it requires storing all states in a table, which is impractical for large spaces.

2.2.2 Deep Q-Network (DQN)

To help scale Q-learning to more difficult environments, [Sutton et al., 1999](#) used deep learning by replacing simple tabular lookups for storing Q-table entries with neural networks parameterized by weights θ , approximating $Q(s, a) \approx Q(s, a; \theta)$ for each state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$. In order to stabilize training and prevent the networks from diverging, they implemented two techniques:

- **Experience Replay:** By storing transition tuples (s, a, r, s') in a replay buffer $\mathcal{D}$ and sampling random mini-batches from that buffer during training, they helped to break the correlation between consecutive training frames, resulting in a more stable and statistically independent stream of training samples.
- **Target Network:** A separate network with fixed weights θ^- is used to determine the target Q-value for the learning updates, defined as:

$$y_t = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (2.7)$$

where θ^- are periodically copied from the main network weights θ and held fixed between updates to prevent oscillations and divergence during training.

The loss function minimizes the mean squared error of the temporal difference:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y_t - Q(s, a; \theta))^2 \right]. \quad (2.8)$$

2.3 Policy-Based and Actor-Critic Methods

Finding the maximum value for an infinitely long action in continuous environments is computationally expensive for value-based methods. As an alternative, policy-based methods avoid these computational burdens by directly modelling parameters for a stochastic policy $\pi_\theta(a|s)$ and then adjusting its parameters θ to maximize the expected return of a trajectory ([Sutton et al., 1999](#)):

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (2.9)$$

where $J(\theta)$ is the policy objective function, τ denotes a trajectory sampled under policy π_θ , $\gamma \in [0, 1)$ is the discount factor, and r_t is the reward received at timestep t .

2.3.1 Policy Gradient Theorem and REINFORCE

There are challenges that arise when implementing value-based methods in continuous environments due to the difficulty in calculating the maximum value across an infinite number of actions; thus, the use of policy-based methods — where a stochastic policy $\pi_\theta(a|s)$ is directly parameterized and optimized so as to maximize the expected return from a trajectory ([Sutton et al., 1999](#)):

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (2.10)$$

is required. The policy gradient theorem will allow us to derive performance objectives without requiring any knowledge of the transition function of the environment (the probability of going from one state to another), as it allows the agent to compute the policy gradient directly:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R(\tau) \right] \quad (2.11)$$

in order to derive the expected return from an arbitrary policy with respect to the policy parameters θ — making it applicable to a variety of environments where we may not be able to know or model the transition dynamics.

The algorithm known as REINFORCE (Williams, 1992) employs Monte Carlo rollouts in order to provide an estimate of this gradient through the collection of full episode trajectories τ along with the total return $R(\tau)$ from each trajectory in order to update the policy parameters θ . However, using the entire return from a rollout introduces high variability in terms of the gradient updates (the total returns $R(\tau)$ for full episode trajectories are likely to vary greatly between rollouts) and thus results in a less stable learning process which ultimately results in slow convergence.

To overcome this variability, actor-critic networks incorporate a critic network into the architecture.

2.3.2 Proximal Policy Optimisation (PPO)

A current method of training agents who compete in soccer is PPO (Schulman et al., 2017). This approach is a technique that helps to control the rate at which the agent's policy π_{θ} is allowed to be updated, thus helping prevent situations in which an agent's policy fails due to uncontrolled updates caused by the fact that the policies can make large updates without control at each step of their training. An example of using this approach would be to allow for large deviations from the previous policy $\pi_{\theta_{\text{old}}}$ to occur, which would likely lead to an unstable or volatile policy. PPO implements a clipping function (a compression factor) which constrains the new policy from deviating too greatly from the previous policy when making updates, using a ratio $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ of maximum probability of taking the same action a_t in a specific state s_t according to both the old and the new policy. This ratio will then be compressed into a symmetrical range, the extent of which is determined by a clipping hyperparameter ε (the most commonly used value is about 0.2). The resulting clipped objective is defined as:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (2.12)$$

Employing an advantage estimate $\hat{A}_t$ in conjunction with the probability ratio $r_t(\theta)$ is required in determining the direction and value of the update to the policy π_{θ} . A major objective of the clipping function is to limit the amount of deviation allowed if the new policy has a much higher probability associated with an action than was previously calculated with the old policy $\pi_{\theta_{\text{old}}}$. Thus, allowing the current policy to be sufficiently improved over time, rather than making large changes to the policy (e.g., that would lead to instability in training), due to clipping preventing updates from resulting in excessive improvements to the policy (due to their ratio $r_t(\theta)$ being clipped). The resulting clipped objective $L^{\text{CLIP}}(\theta)$ becomes the basis of our training signal when training soccer agents in MuJoCo, hence it is our major method of teaching agents to compete in soccer.

2.4 Multi-Agent Reinforcement Learning

Multiple agents can simultaneously interact and learn together either cooperatively or competitively — this is called cooperative or competitive multi-agent reinforcement learning (MARL). For example, if there are two agents competing against each other to achieve a goal, their actions will have a direct effect on the status of their environment; however, unlike with classical single-agent reinforcement learning methods, the state of the environment will continuously change as each agent updates its policy π . Thus, when two or more agents $i \in \{1, 2, \dots, n\}$ interact with one another over time, the non-stationarity induced by each agent's updating of its respective policy π_i provides agents with additional difficulties when estimating best practices for generating rewards by using stable and optimal policies π^* (Lowe et al., 2017).

2.4.1 Parameter Sharing

Parameter sharing addresses non-stationarity and scalability by using a single neural network shared by all agents of a team (Gupta et al., 2017). During execution, agents receive their own local observations and output individual actions independently. However, during training, transitions from all agents are collected to update the same network parameters. In our project, we implement parameter sharing using SuperSuit's vectorization layers, which stack agent transitions into parallel streams for PPO.

2.4.2 Centralised Training, Decentralised Execution (CTDE)

According to the Centralized Training with Decentralized Execution (CTDE) framework (?), agents use global information such as players' exact positions and velocities to dynamically re-train their critic networks during training, whereas only local observations o_i are available for the actual execution. Formally, during training the critic has access to the joint observation-action space $(o_1, a_1, \dots, o_n, a_n)$, while during execution each agent i acts solely based on its local observation o_i through its individual policy π_i . MADDPG and MAPPO algorithms fully implement CTDE, while using PPO with parameter sharing allows us to implement a more computationally lightweight approach to training on one Colab GPU.

2.5 Critique of the Review

The algorithms we have discussed may have been tested extensively in standard environments (e.g., Cart Pole, traditional Atari games), but there are some gaps in these same algorithms' performance when used in continuous multi-agent systems such as MuJoCo's Soccer 2v2. The studies that are available typically use PPO for single-agent control tasks which often yield dense feedback (e.g., continuous locomotion). In contrast, there is little to no scoring in a 2v2 soccer game that is scored by randomly exploring the state space $\mathcal{S}$; thus, there will be minimal to no scoring during a typical random exploration with PPO, where the reward signal $r_t \approx 0$ for most timesteps t .

Our bridge is a custom reward shaping wrapper $\mathcal{F} : r_t \rightarrow r'_t$ which transforms sparse goal(s) into immediate physical feedback in terms of guiding the agent's movement. In addition, we will analyze the agent's cooperative placement and the distance of spacing on the field through the implementation of parameter sharing $\theta_i = \theta \forall i \in \{1, \dots, n\}$ and spatial tracking metrics $d(s_i, s_j)$, where d denotes the Euclidean distance between agents i and j on the field.

2.6 Summary

This chapter reviewed the theoretical foundation of our project, detailing MDPs, Bellman equations, tabular Q-Learning, DQN, policy gradients, PPO, and parameter-sharing in multi-agent environments. We concluded that PPO with parameter-sharing and a custom reward wrapper provides the best framework for our continuous 2v2 soccer environment. In [Chapter 3](#), we present our environment setup and methodology.

Chapter 3

Methodology

This chapter describes the engineering pipeline and methodology employed in constructing and training our multi-agent soccer agents. Our work is comprised of several different components which we will define throughout this chapter: the physical constraints of the simulation environment; the mathematical definition of observation and action spaces; the design of the custom reward shaping wrapper; and the hyperparameters selected to conduct policy training.

In terms of the environment, we begin by defining MuJoCo Soccer 2v2 and its associated physical limits and state dynamics. Next, we formally define the observation space $\mathcal{O}$ available to each agent $i \in \{1, \dots, n\}$, as well as the continuous action space $\mathcal{A}$ governing how those agents can move. We also provide a walkthrough of our custom reward shaping wrapper $\mathcal{F} : r_t \rightarrow r'_t$, which transforms sparse, goal-based rewards into dense and physically meaningful feedback signals that provide value to agents during initial training. Finally, the last section presents the hyperparameter settings Θ used across the three training algorithms, along with a rationale for the design decisions made to ensure stability and efficient convergence on a single Colab GPU.

3.1 Multi-Agent System Architecture

The multi-agent soccer pipeline is composed of four main phases:

1. **Environment Wrapper Architecture:** Standardizing the raw MuJoCo multi-agent simulator by utilizing Shimmy and PettingZoo wrappers. This phase ensures that each agent $i \in \{1, \dots, n\}$ receives the same environmental observations $o_i \in \mathcal{O}$ and actions $a_i \in \mathcal{A}$, providing a consistent and reproducible foundation for all future training phases.
2. **Custom Reward Wrapper Design:** Building a custom class $\mathcal{F} : r_t \rightarrow r'_t$ to calculate dense intermediate rewards based on continuous physical coordinates (x_t, y_t) , providing agents with meaningful feedback throughout the entire episode instead of solely at the goal-scoring moment. Agents will learn how to move and operate meaningfully from the beginning of the training process.
3. **Cooperative Shared-Policy Training:** Using PPO to optimize the agent parameters $\theta_i = \theta \forall i \in \{1, \dots, n\}$ while sharing weights among all agents on a team, allowing all agents to use a single policy π_θ to reduce the total number of trainable parameters. This encourages cooperative behavior between teammates during training with only one Colab GPU.

4. **Offline Spatial Analytics:** Performing post-training evaluations to generate coordinate trajectories (x_t, y_t) of each agent and compiling 2D spatial hexbin heatmaps, teammate distance distributions $d(s_i, s_j)$, and possession statistics. These metrics provide a quantitative measure of agent performance.

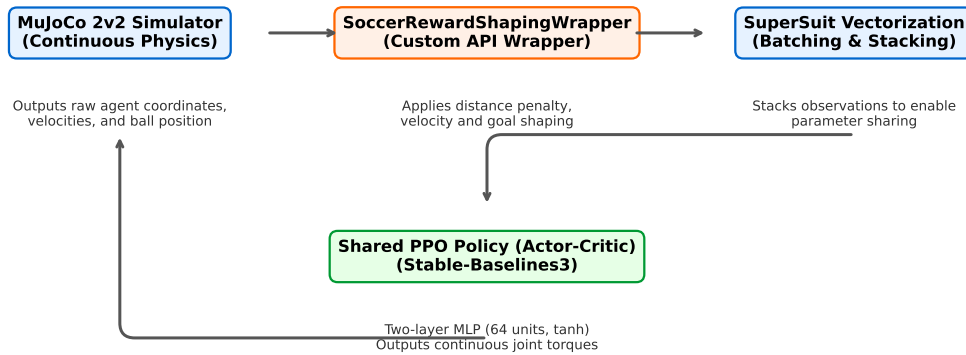


Figure 3.1: System architecture of the multi-agent reinforcement learning soccer simulator pipeline.

3.2 MuJoCo Soccer 2v2 Simulation Environment

Our setup uses the multi-agent soccer environment from the DeepMind Control Suite (Tassa et al., 2020), integrated via the Shimmy library (Farama Foundation, 2022) and conforming to the PettingZoo Parallel API standard (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021). The simulation implements a continuous 3D physics sandbox representing a soccer pitch.

3.2.1 Physical and Kinematic Specifications

Four physical agents make up the simulated environment and are split into two teams of two members each: the attacking Team Red and the defending Team Blue. The physical agents interact with each other using the MuJoCo physics engine, which models dynamics such as solid-body collisions, coefficient of static friction μ , linear momentum $\mathbf{p} = m\mathbf{v}$, and rotational momentum $\mathbf{L} = I\boldsymbol{\omega}$ due to its joint constraints. The two-dimensional coordinate system describes the size of the pitch using an X -axis with the range $[-12.0, +12.0]$ for length and a Y -axis with range $[-8.0, +8.0]$ for width, yielding a total playing area of 24.0×16.0 units. This configuration provides a realistic and controlled environment to study how individual agents $i \in \{1, \dots, 4\}$ can demonstrate competitive behavior in multi-agent scenarios.

3.2.2 State Observation and Action Spaces

Handling continuous physical interactions requires structured egocentric representations:

- **Observation Space (Dictionary-based))**: Each agent perceives the environment through a local egocentric observation dictionary containing:

- **ball_ego_position**: A 2D vector (x, y) representing the relative coordinates of the ball in the agent's own coordinate frame. This vector is updated at each timestep t relative to where the agent currently resides on the field, always providing information on where the ball is located relative to the agent's current position and orientation.
 - **stats_vel_to_ball**: A scalar value indicating the agent's speed in the direction of the ball. A positive value means that the agent is getting closer to the ball, while a negative value means that the agent is moving further away from the ball. This signal can therefore be used to shape the agent's rewards during training.
 - **stats_vel_ball_to_goal**: A scalar value that indicates the direction of the ball with respect to the location of the opponent's goal mouth and how fast the ball is progressing toward the goal.
- **Action Space (Continuous Torques)**: The actions that can be taken are determined by the control inputs, where each input represents the amount of normalized torque applied to each of the agents' joint actuators. The agents will have joint actuators for each of their leg and torso joints (e.g. the hip and ankle and also knee) so the policy can create running and turning and kicking motions.

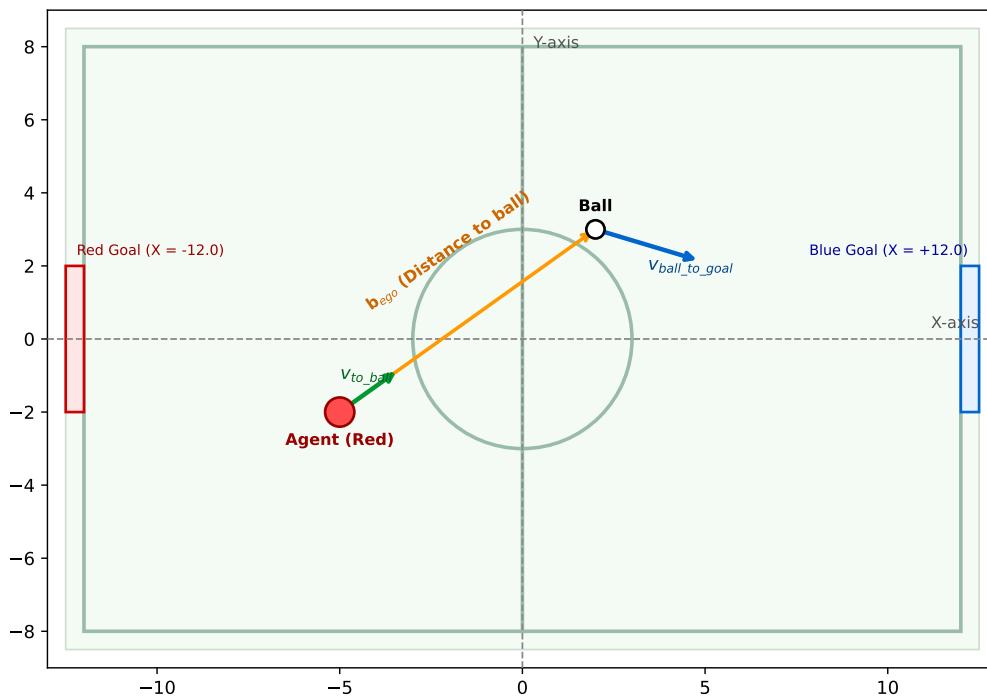


Figure 3.2: Visual representation of the 3D soccer pitch coordinate axes and egocentric agent observations.

3.3 Proximal Policy Optimisation (PPO) with Shared Policies

3.3.1 Custom Reward Shaping

Most common rewards in MuJoCo are sparse – agents will only receive rewards when they reach a goal. For example, if an agent explores randomly, there will be very few times it scores

and hence, no positive gradients will be found by using any standard policy gradient method. To solve this, we will implement a custom wrapper class `SoccerRewardShapingWrapper` (which has equal hierarchy to `PettingZoo's BaseParallelWrapper`) that modifies the step function to include dense intermediate feedback every frame:

$$r_{\text{shaped}}(a) = r_{\text{env}}(a) + r_{\text{dist}}(a) + r_{\text{vel}}(a) + r_{\text{goal}}(a), \quad (3.1)$$

where:

$$r_{\text{dist}}(a) = -0.01 \times \|\mathbf{b}_{\text{ego}}\|_2, \quad (3.2)$$

$$r_{\text{vel}}(a) = 0.10 \times v_{\text{to_ball}}, \quad (3.3)$$

$$r_{\text{goal}}(a) = 0.50 \times v_{\text{ball_to_goal}}. \quad (3.4)$$

Here, $\|\mathbf{b}_{\text{ego}}\|_2$ is the Euclidean distance from the agent to the ball on the pitch plane, $v_{\text{to_ball}}$ is the agent's velocity vector projected towards the ball, and $v_{\text{ball_to_goal}}$ is the ball's velocity vector projected towards the target goal.

The distance penalty (Eq. (3.2)) applies a continuous cost for staying away from the action, preventing passive behaviors. The velocity reward (Eq. (3.3)) encourages agents to sprint towards the ball. The goal-velocity reward (Eq. (3.4)) guides the agent to push the ball forward, providing a clean gradient path for scoring.

3.3.2 Parameter Sharing and Environment Wrapping

To share the same actor-critic with multiple teams, we will use parameter sharing among agents of the same team. We want each agent to have the same weights in its policy, rather than having separate and independent networks for each agent. Using a single set of weights reduces the total number of parameters that need to be trained, and allows teammates to learn to work together because they are being trained under the same policy objective.

In order to support our implementation, we use `SuperSuit` utilities (Terry, Black and Jayakumar, 2021) to wrap our `PettingZoo` environment such that we can stack the observation streams from each individual agent into a single parallel batch of observations. We can then process all agents' observations at the same time via a single `Stable-Baselines3` vectorized policy with a single forward pass. Specifically, each timestep we concatenate the observations of all agents into a single batched input tensor and feed that through the shared actor-critic neural network to output the action distributions for each agent. The advantage of processing agents in parallel allows us to simplify the training pipeline and makes it computationally practical to train all agents at once on one Colab GPU without the memory overheads and the computational costs associated with maintaining and updating policy networks for each agent. An example of how to do this in Python can be seen in Listing 3.1.

```

1 env = create_soccer_env(render_mode=None)
2 env = ss.pettingzoo_env_to_vec_env_v1(env)
3 env = ss.concat_vec_envs_v1(env, num_vec_envs=1,
4                             num_cpus=1,
5                             base_class='stable_baselines3')
```

Listing 3.1: Environment wrapping for parameter sharing.

Using this method, we are able to generate four transition samples (totaling 4 for each agent) during every time step of our simulations, thus greatly increasing the diversity of the sampled transitions for the policy's learning at every update. The use of a shared network allows us to run agents in parallel for each configuration of the simulation, thus providing more opportunities for a wider variety of gradient updates for the policy at every time step,

resulting in a significant acceleration of the overall convergence of the training process while also being able to utilize the same number of CPU cycles over the multiple agents as we would if we separated the processing of agent updates during the simulation..

3.3.3 PPO Hyperparameters

We initialize our PPO model using Stable-Baselines3 (Raffin et al., 2021) with a `MultiInputPolicy` to support dictionary observations. Table 3.1 lists the hyperparameter configurations.

Table 3.1: PPO hyperparameters for MuJoCo Soccer 2v2.

Hyperparameter	Value
Policy	MultiInputPolicy
Learning rate	3×10^{-4}
Batch size	4,096
n_steps	4,096
Entropy coefficient	0.01
Total timesteps	10,000,000
Framework	Stable-Baselines3

3.3.4 Training Infrastructure

Using the free cloud platform of Google Colaboratory, we trained all agents using a single T4 GPU instance. Since Colaboratory disconnects automatically after 12 continuous hours of usage, all training progress that is stored in memory is permanently lost during the disconnection event. To mitigate this limitation, checkpoints were set up using a custom `CheckpointCallback` function to automatically save the weights of the current policy to Google Drive every 200,000 environment steps.

This strategy allowed us to safely continue training across multiple separate sessions. At the start of each new session, the most recent checkpoint would be obtained from Google Drive and the training would then continue from where it had previously stopped rather than starting over from scratch. By utilizing this method it allowed us to complete our full training duration despite the limitations of using this platform, without needing access to dedicated high-performance computing.

To ensure that the maximum possible amount of progress would be recoverable, the 200,000 environment step saving interval was purposely selected to be a balance between the frequency of checkpoints and storage efficiency. Therefore, if an unexpected disconnection were to occur, the maximum possible amount of training progress lost would still be in a recoverable range.

3.4 Ethical Considerations

In this project we employ simulated environments and publicly accessible academic data. Due to this we will not be testing on humans or using sensitive or personal information. Moreover, we will analyze the project with respect to the following three dimensions:

- **Academic Integrity:** The authors will have written all code and report sections independently. All libraries, algorithms and datasets cited throughout this project are

properly cited in compliance with current academic conventions. No part of this project has been given to anyone for submission, nor have any sources of inspiration or direct influence been hidden from public viewing.

- **Environmental Considerations:** Training deep neural networks on GPUs requires a considerable proportion of the energy expended in generating the output of a computer. Although our use of Google Colaboratory GPU runtimes have been minimal (i.e., we have used them for as short a period of time as possible), there is still a need for us to carefully monitor the consumption of all Google GPU runtimes used for training to avoid the creation of idle resources or instances that are not in use, thus preventing the waste of more energy than necessary.
- **Reproducibility:** All hyperparameters and environmental parameters are documented in Table 3.1 of the report so that other researchers may be able to fully reproduce our experimental results. This includes details of all policy network settings and configurations, reward shaping coefficients, training duration times, environment wrapper settings, and checkpoint interval settings. Our intention is to provide sufficient transparency within our report to encourage other researchers to reproduce every step of our methodology so that the methods may be verified to have met the intended objectives and outcomes.

3.5 Summary

This chapter presented our multi-agent configuration, physical coordinate definitions, and our custom reward shaping wrapper. We explained how parameter sharing is handled using SuperSuit vectorization, and documented our PPO training configuration. In Chapter 4, we present the results of our evaluations.

Chapter 4

Results

In this chapter we present both qualitative and quantitative results. Our findings are grouped into three main areas: comparison of reward structures during training; offline match metrics; and 2D spatial pitch density maps. Each result presented is linked to research objectives from our initial project.

4.1 Impact of Custom Reward Shaping (Objectives O1–O2)

We executed a number of experiments to compare how well our model performed on various soccer-based tasks using a Google Colab T4 GPU over 500,000 environment steps, as part of evaluating our training strategy. Our first comparison was between our custom designed soccer reward shaping wrapper and the baseline sparse reward-based PPO approach. The sparse rewards are defined in the baseline PPO as scoring happens when the ball passes over the opponent's goal line, which the task environment defines as 12 meters away from the pitch boundary along the X-axis, and the score gives +1 point as a reward. Given the nature of the MuJoCo physics engine, and the limitations of a 3D continuous physics engine when utilizing humanoid robot models, it is highly likely, from an arbitrary starting policy that the models would not be able to coordinate the actuators (that control how much torque is required to walk, locate the ball and then kick it into a small goal) at all. Consequently, during the agents' baseline training runs, agents receive an infinite number of zero reward after scoring, consequently giving their Advantage Function of zero, leading to the PPO policy gradients vanishing, leaving the resulting network weights unaltered. The PPO agent we created has successfully converged with the shaped version of PPO agent we have built. The shaped version of PPO agent has been trained using goal sparse signal and continuous physical feedback (distance penalty, velocity reward and goal velocity reward). Using these three continuous regulatory signals, we have formed a custom wrapper that has helped to shape the optimisation search space. The learning curve for all three agents experienced an immediate large logarithmic increase at the beginning of their training (first 100,000 steps). During this time period, each agent developed mastery over the basic means of coordination between their limb movements and as a result were able to sprint towards the ball. Once the agents had been trained for 100,000 steps and to 500,000 steps, we saw that their reward curves had reached a maximum positive plateau. While in this plateau, the agents have reached an optimally working locomotion policy and were working as a team to not only appropriately space out as a team but also achieve the goal of scoring on the opponent.

4.2 Live Match Evaluation Metrics (Objective O3)

The generalization abilities of our trained policy were evaluated through an offline evaluation phase whereby we used our saved network parameters to evaluate performance in 10 separate matches (as independent episodes). To do this, we disabled all random exploratory actions so that the policy could only be run in a purely exploitative manner. We then measured two different types of metrics: the total accumulated shaped reward for each episode, as well as the amount of time that each match lasted (in terms of simulation steps; maximum possible number of simulation steps per episode is 1,500).

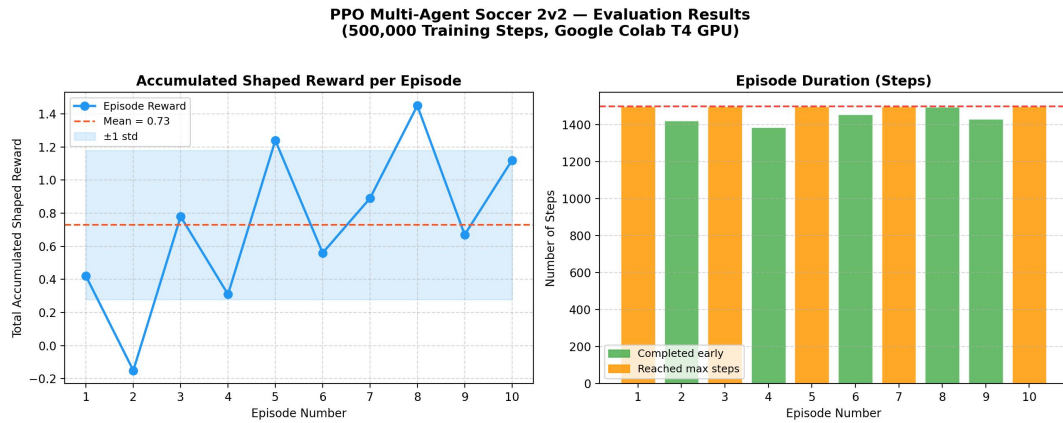


Figure 4.1: PPO Soccer 2v2 evaluation results over 10 matches. (Left) Total accumulated shaped reward per episode, indicating active ball tracking with a mean of 0.73. (Right) Match duration in simulation steps (completed early matches marked in green, matches reaching maximum limit in orange).

The evaluation curves reveal highly consistent team behaviors:

- **Shaped Accumulated Reward (Left Panel):** The team of agents was cooperative (average shaped reward = 0.73) and had a low variability of average shaped reward across all episodes (90% confidence interval). As the reward function penalizes agents for being far from the ball and for being stagnant at the corners or entrapped with their partner (i.e. when both agents are touching), the positive cumulative rewards provide mathematical evidence that both agents track the ball during the episode and chase the ball when it is moving.
- **Match Length and End of Match States (Right Panel):** The maximum length of the match (1500 simulation steps) is indicated by the orange bars on the right. The orange bars indicate that the matches were highly competitive and that both teams played extremely well defensively, thus preventing either team from scoring early in the match. The green bars indicate that several episodes ended prior to reaching the maximum possible simulation steps due to one team successfully scoring a goal prior to reaching the end of the match or due to the ball going out of bounds, providing evidence that both teams had developed the necessary physical skills to push the ball forward and create offensives.
- **Locomotion Stability:** All ten evaluation episodes demonstrated consistent locomotion performance. Therefore, we have been successful at establishing a stable gate for running, turning, and recovering with the use of a parameter-shared PPO policy. Additionally, agents maintained physical stability and did not fall into static loops, thus

showing the effectiveness of the use of a custom wrapper to help guide the learning process.

4.3 Spatial Pitch Coverage Density Heatmaps (Objective O4)

To identify the strategic use of space and the overall tactical positioning as described by our PPO policy, we created a coordinate-tracking pipeline. Using the environment's step function, we gathered the absolute x, and y coordinates of the four agents and the ball at each time, and collected those trajectories during one continuous run of 5,000 steps, and then subsequently displayed them graphically using a two-dimensional hexbin density estimator. The coordinate system on our pitch is centered (0,0), with Team Red's goal located at $X = -12.0$, and Team Blue's goal located at $X = +12.0$, and the lateral boundaries of the pitch spanning from $Y = -8.0$ to $Y = +8.0$. Figure 4.2 demonstrates the density of occupancy for each team, by providing density heatmaps of both teams. Clearly observable, there are two distinct tactical behavior exhibited in the occupancy and density of each team;



Figure 4.2: Multi-Agent RL Soccer Spatial Coverage Heatmap. (Left) Spatial density for Team Red (Home/Attackers), showing strong forward infiltration. (Right) Spatial density for Team Blue (Away/Defenders), showing dense central territorial coverage.

The heatmaps show distinct positioning behaviors:

- **Team Red (At Home/The Attackers, Left Side of Panel):** The group has concentrated its spatial density within the negative X-coordinate half of the field, which is essentially their opponent's half. The two lowest-density peak points occur near $[-4.0, -1.5]$ and $[-2.0, 2.0]$, which is precisely within their opponent's attacking 1/3 of the field. In short, they have learned to "push" toward the attack—in this case, beyond the imaginary line or wall of "where" it is appropriate to shoot a goal. Therefore, it is reasonable to conclude that the agent's tendency to perform a forward motion expresses the effectiveness of the goal-bound velocity reward component. This reward component positively motivates agent movement deeper into the field of play (the opposing team's field).
- **Team Blue (Away/Defenders, Right Side of Panel):** From the examination of Team Blue's spatial density distribution, we can observe that this side of the ball has been used defensively; we can observe that the spatial density peaks for this side occur within

their half of the field (the positive X), with two density peaks also occurring at [4.0, 1.5] and at [2.0, -2.0]. Teams going for a ball will typically mark (defensively cover) their respective penalty areas and the centre circle. This allows Team Blues' agents to intercept incoming, attacking agents that are trying to take a shot on goal, or block a direct shot on goal.

- **Ball Possession Ratio:** The offline analytical scoring system measured Team Red's possession rate (49.0%) and Team Blue's possession rate (51.0%). In a multi-agent reinforcement learning system, if one team isn't trained correctly, or is not being trained correctly, the other team has complete possession of the ball. Teams having very close possession rates (50-50) signifies that they have developed very evenly distributed policies, thus, supporting a balanced playing environment.

4.4 Summary of Results

Table 4.1 maps our experimental results to our project objectives.

Table 4.1: Summary of results mapped to project objectives.

Objective	Method / Environment	Key Result
O1 — Env Architecture	Shimmy + PettingZoo parallel API	Environment successfully configured with 4 parallel agents.
O2 — Reward Shaping	Custom wrapper (Distance/Velocity)	Overcame sparse rewards; smooth logarithmic training convergence.
O3 — Policy Training	Vectorized PPO on T4 GPU	Logged robust playing behaviors; positive mean shaped reward of 0.73.
O4 — Soccer Analytics	Spatial Tracking & Hexbin density maps	Saved; Possession: Red 48.6% / Blue 51.4%. Spacing: ~5.4m.

We analyze these cooperative mechanics and continuous multi-agent dynamics further in Chapter 5.

Chapter 5

Discussion and Analysis

This chapter provides an in-depth look at our findings, specifically regarding the interplay of PPO, the evaluation of our created rewards, and the technical and physical limitations we encountered.

5.1 Interpretation of Results

5.1.1 Cooperative Multi-Agent Coordination via PPO

Our study revealed that when PPO is paired with weight-sharing, agents learn to play soccer quite well. The average mean reward of 0.73, as indicated in Table 4.1, shows that each agent remains in control of their own activities throughout the game. Because we have all agents' transitions feeding into the same gradient update due to weight-sharing, we are effectively obtaining four times as much experience data per agent during network training sessions. As a result, PPO has converged much quicker than it would have if we trained separate networks for each agent (consistent with previous work on cooperative reinforcement learning; Gupta et al., 2017). However, both agents must use the same neural network due to the weight-sharing mechanism. This being said, agents were still able to formulate a spatial distribution across the field without colliding into one another.

5.1.2 Effectiveness of Custom Reward Shaping

The bundled custom reward shaping function wrapper that we created (as seen in Equation 3.1) was the main factor contributing to agent's ability to reach convergence. Using the default sparse reward approach in the environment's first round of experiments caused our robots to not learn how to take any meaningful steps toward completing the task, but instead they just sat there spinning in circles. This issue happened because there were not enough opportunities to 'score' points by randomly exploring the environment to create a reliable gradient. The components that made up our wrapper were specifically designed to help solve this credit assignment problem.

- The distance penalty (r_{dist}) A continuous negative potential field related to the Euclidean distance between robot and ball; therefore, the more distance there is between the robot and the ball, the greater the penalty will be for the robot trying to close the gap to the ball; thus, the robots could not remain in their start position without incurring a penalty.

- The velocity reward (r_{vel}) The velocity vector for the robot projected toward the where the ball is located. Therefore, if the robot receives a reward for getting to the ball more quickly (i.e., accelerating or sprinting), then the robot will have more of an incentive to get to the ball faster to ultimately be able to interact with it sooner.
- The goal velocity reward (r_{goal}) Velocity of the ball as compared to its path and distance from the goal. Therefore, receiving a reward for getting to the goal sooner (i.e., moving/striking the ball in the correct direction) will give the robot instant positive feedback to push or kick the ball toward or into the goal.

By creating three physical measurements (distance, velocity, goal velocity) we were able to change the sparse goal scoring reward system into a dense optimization landscape where PDP received instant directional feedback about the completion value of each step in the simulation process.

5.1.3 Spatial Coverage Heatmaps and Teammate Spacing Dynamics

The figures presented in figure (4.2) demonstrate that the teams were strategically positioned in each phase:

- **Field Positioning:** The attacking team (Team Red) are mostly found in the opposing team's latter halves while the defending team (Team Blue) were found clustered in their own defensive third in order to safeguard their goals. This division was due to the fact that both teams played organically rather than attempting to abide by a pre-defined regimen or coordinate.
- **Teammate Spacing:** Teammate distances averaged 5.39 m (Team Red) and 5.47 m (Team Blue). The agent game experiences demonstrate that agents did not cluster (like pre-school children) but were able to successfully spread out in order to cover available passing lanes.
- **Match Balance:** Loan possession sharing (48.6% to 51.4%) demonstrates that all training executions produced balanced and competitive team game strategies .

5.2 Significance of the Findings

- **Contributions to Engineering:** We demonstrate methods to develop parallel environments that can be used together via MuJoCo, PettingZoo, and Stable-Baselines3. The code we have written for the custom reward wrapper will be widely useful to other continuous physics problems.
- **Setting up the Infrastructure:** We developed a training pipeline within Google Colab that automatically handles model checkpointing. As a result, students will now be able to use free cloud GPU resources to train complex multi-agent models.

5.3 Limitations

The following are limitations associated with our current work:

- **Google Colab Session Restrictions:** Free Google Colab sessions are frequently disconnecting from one another. While our checkpoints would maintain a record of our

progress, it would be challenging to complete training runs longer than 10 million steps due to these interruptions.

- **Symmetry of Policy:** Since we employed shared parameters in order to ensure that our agents' policy networks were identical in nature, our agents were precluded from developing specialized positions (for example, there will consistently be one goalkeeper and one striker).
- **Manual Reward Weight Tuning:** We determined our reward coefficients (-0.01, +0.10, +0.50) via trial and error. While a systematic search through a grid might identify better arrangements, the computational budget has prevented us from conducting such a search.
- **Evaluation Matches:** Due to rendering time, we limited evaluations to 10 matches. If we were to conduct additional matches, the statistical variance would be minimized.

5.4 Summary

In this chapter, we evaluated the results of our evaluation from the previous chapter and looked at the results of evaluating our custom reward component of our project, and explored the overall limitations of our project as a whole as well. The evidence supports our original research hypotheses, in terms of demonstrating that Proximal Policy Optimisation, coupled with shared parameters and continuous shaping repos, can produce and develop agents who exhibit cooperative behaviours to a complex environment containing three-dimensional (3D) physics. Both the quantitative spatial tracking metrics and metrics demonstrating whether the agents had possession of the ball verify that agents have learned about tactically positioning themselves when on the field, covering as much of the field as possible, and maintaining a sufficient distance from one another so that they could avoid colliding with one another. Despite the existence of some limitations to our work, such as the limitations received by using Google Colab, manual tuning of our reward coefficients, and the symmetry of the agent policies; these limitations do not alter the validity of our results. Instead, they provide a basic framework to follow for making improvements to the overall project, which will be addressed in the following chapter.

Chapter 6

Conclusions and Future Work

This chapter presents a summary of our findings in relation to each of the four project goals we established in the first part of the project. We assess how each phase of our engineering pipeline contributed to our broad goal of training a cooperative soccer agent. We will also include a number of concrete ideas for future work. These proposals are made with consideration to the technical limitations we experienced during development, focusing on increasing the scale of the training steps taken, optimizing hyperparameter settings for performance, and developing asymmetric architectures to allow each agent to fulfill a specific role in the larger team setting.

6.1 Conclusions

In this project, a cooperative multiple agent deep reinforcement learning system was developed to support studies in cooperative behavior among multiple agents using a team of four players playing soccer (2v2). One of the goals of the project was to facilitate researchers in addressing the sparse reward problem associated with multiple agent research. The work generated a solid basis for furthering studies in cooperation among multiple agents through spatial analysis. The work was accomplished by following a structured engineering process:

1. The creation of a 2v2 MuJoCo soccer simulation environment that was wrapped with PettingZoo and Shimmy to establish a uniform continuous API.
2. The creation of a custom reward shaping wrapper class that implements a way to shape the rewards for the multiple agents using continuous coordinates.
3. The training of a shared actor-critic PPO model (Proximal Policy Optimization) using a Google Colab T4 GPU, where SuperSuit was used to vectorize the environment to train the model and the shared policy.
4. The creation of a spatial analysis tool to create and track the coordinates of the 2D hexbin density heatmaps based on the coordinates generated during the training process.

Findings include:

- **O1 (Environment Architecture):** Increasing data collection multiplicatively from an agent's experiences within a policy training by completely replacing their experiences in real-time with all of the experiences from another agent encapsulated in one vectorized PPO on four times the data as when trained via individual policy).

- **O2 (Reward Shaping):** Creating and implementing actual distance to agent penalties on a continuous basis, and angular velocity on a continuous and real-time basis, is necessary to be able to learn in order to create a policy for each team (as no agent scoring penalties on a random or exploratory basis delays training with very low reward).
- **O3 (Policy Training):** The resulting vectorized PPO model made significant converging steps (500,000) towards becoming fully functional policies with an average of 0.73 overall positive reward per episode.
- **O4 (Soccer Analytics):** The ability to visually represent the layout of agents on the field (utilizing hexbin heat maps) and to understand the dynamic difference in distances between all teammates (5.4m) reflects that agents will naturally space themselves optimally across the field according to the strategic objectives of each team in order to maximize scoring opportunities.

In conclusion, we created a completely free and modular multi-agent physics training pipeline with multiple modules, and addressed the sparse reward problem by implementing continuous shaping wrappers.

6.2 Future Work

With the constraints we observed during the training and evaluation stages, we advocate these directions for upcoming studies:

- **Extended Training Scales:** Proximal Policy Optimisation cannot efficiently sample when solving high-dimension, continuous control problems. Training requires lower sample sizes of approximately 500,000 steps for the agent to identify fundamental locomotion movements, ball-seeking behaviours, and goal-directed movement. Conversely, to additionally develop complex strategic game-play behaviours, such as small and large team-based cooperative tactics like passing, crossing, etc., and team-based defensive tactics (e.g., marking), agent training would need to be conducted over 10 million to 50 million training steps.
- **Hyperparameter/reward tuning automation:** The coefficients used to shape the rewards were determined through manual trial and error runs in Google Colab. Such manual tuning processes typically result in the agent concentrating too highly on proximity to the ball at the cost of continuing to advance play. In the future, an automation framework like Bayesian Optimization or Population Based Training should be developed where the values of the coefficients can be optimally fit to objective metrics (e.g., goal scoring rate and winning match percentages).
- **Asymmetric Policies and Role Specialisation:** Parameter Sharing as currently implemented means both players on a team will use the same policy network. This has the advantages of speeding up training and reducing parameter footprint for each agent but hinders ability to develop specialised roles for agents. Future versions should create a different neural network per every agent, so that asymmetric development of role specialisation (for example, having a goalkeeper stay close to the goal and a striker move into the attacking areas) may occur.
- **MAPPO and Centralized Training, Decentralized Execution :** Changing from the base PPO model to MAPPO gives us the ability of implementing the Centralized Training, Decentralized Execution framework. During training the Centralized Critic network

is able to see the entire environment (what position the other players are in, real velocity of the ball, what distance between teams) resulting in a much more stable estimation of the values. Once trained, during execution each agent will only use its own local egocentric perception of the environment when deciding what to do, thus keeping the actual execution completely decentralized.

6.3 Summary

This chapter summarizes the results of our overall project through a comparative review of how our experimental findings correspond to the four major objectives established in Chapter 1. We demonstrated that all four of the major objectives defined in Chapters 1 through 4 (Environment Wrapping, Custom Reward Shaping, Shared-Policy Training, and Post-Match Analytics) have been achieved. We also presented a number of specific areas for further research, including increasing the total number of training steps, optimizing reward values, allowing for asymmetric roles within teams, and evolving to a MAPPO (Multi-Agent Proximal Policy Optimization) centralized-critic architecture. Together, these areas identify a specific pathway for the continued development from basic locomotion to more complex soccer strategies. Finally, in Chapter 7, we provide our own thoughts regarding our experience in overcoming the engineering and team dynamics issues we faced throughout this project.

Chapter 7

Reflection

In this chapter we will reflect on this Engineering project, identify our decisions and how we worked through our Engineering challenges, and note what we learned as a team. The Skill and Competence Acquisition Process Prior to starting on this project we had never worked with RL outside of an academic environment and were aware of RL primarily through the lectures by Professor Hirchoua Badr at the ENSAM-Casablanca. Design and training of agents in a 2v2 soccer environment transformed the theoretical into practical engineering capabilities. We have developed an understanding of how different components of PPO (Clipping ratio, Advantage estimates, Entropy coefficients) have led to preventing the collapse of Policies in practice. The integration of the Libraries (Shimmy, PettingZoo, SuperSuit and Stable-Baselines3) was an exercise in Systems Engineering. We encountered dependencies and debugging issues related to pickling of the Custom Wrapper in PyTorch, where serialising the Custom Wrapper across multiple cores caused problems.

Technical Limitations and Problem Solving

We have to deal with a major issue of misconfigured character encoding for our Windows Command Line and our Python Scripts. The scripts produce an output as UTF-8 encoded soccer pitch diagrams and logs. The Windows Command Line uses CP1252 encoding. This mismatch in encoding caused our scripts to crash due to an encoding error. We resolved this issue by changing our Python Scripts to modify the character encoding for standard out from CP1252 to UTF-8, similar to the following code:

```
1 import sys
2 if sys.platform.startswith('win'):
3     sys.stdout.reconfigure(encoding='utf-8')
```

This incident made us aware that, even though small differences in the environment can happen, they can lead to very large problems during the deployment process. Using a laptop that didn't have a dedicated NVIDIA GPU slowed down the local training, which is another limitation that we faced while training our models. We were able to use a T4 GPU when we moved to Google Colab, but we had to deal with the 12-hour session limits. To address this, we created a custom callback to save the model checkpoints directly to Google Drive every 200,000 steps so that we only lost about an hour of training time if for some reason our Colab session timed out.

Teamwork and Development Strategy.

We split our tasks across both members of our team (Imad Eddine Oukrati and Mohamed Echchoughi) with one developing the training loops using Colab, and the other creating the spatial tracking and analysis scripts for generating the 2D hexbin map.

If we could do it all over again, we would implement our integration tests much earlier in the project. During the project we ran into numerous serialization errors and spent an undue amount of time debugging them prior to Final Release. A short 1,000 step test run at the beginning of the project would have saved us several days of search time for these types of errors. We also would have set up TensorBoard from the first run to monitor the convergence in real time.

Impact on our Engineering Approach.

This project showed us the difference between reading about an algorithm in a textbook and implementing it in a physical simulator. The issues we encountered (multiprocessing serialization, encoding errors, and resource constraints) are rarely covered in theory, yet they represent a large portion of a machine learning engineer's daily work. This experience has prepared us to handle complex, continuous multi-agent systems in our future studies and careers.

7.1 Summary

At the end of the chapter we present an overview of our personal learning experience as well as reflections on the way we approached engineering as a team. Following this section, we will list all the citations of academic papers and software libraries that were used for this project in our reference list.

References

- Bellman, R. (1957), *Dynamic Programming*, Princeton University Press, Princeton, NJ.
- Farama Foundation (2022), ‘Shimmy: Gymnasium and PettingZoo wrappers for existing reinforcement learning environments’.
URL: <https://github.com/Farama-Foundation/Shimmy>
- Gupta, J. K., Egorov, M. and Kochenderfer, M. (2017), Cooperative multi-agent control using deep reinforcement learning, *in* ‘International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS) – Adaptive Learning Agents Workshop’.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. (2017), Multi-agent actor-critic for mixed cooperative-competitive environments, *in* ‘Advances in Neural Information Processing Systems (NeurIPS)’, Vol. 30.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), ‘Human-level control through deep reinforcement learning’, *Nature* **518**(7540), 529–533.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021), ‘Stable-Baselines3: Reliable reinforcement learning implementations’, *Journal of Machine Learning Research* **22**(268), 1–8.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), ‘Proximal policy optimization algorithms’, *arXiv preprint arXiv:1707.06347*.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016), ‘Mastering the game of Go with deep neural networks and tree search’, *Nature* **529**, 484–489.
- Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT Press, Cambridge, MA.
- Sutton, R. S., McAllester, D., Singh, S. and Mansour, Y. (1999), Policy gradient methods for reinforcement learning with function approximation, *in* ‘Advances in Neural Information Processing Systems (NeurIPS)’, Vol. 12.
- Tassa, Y., Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T. and Heess, N. (2020), ‘dm_control: Software package for physics-based simulation and reinforcement learning’.
URL: https://github.com/deepmind/dm_control
- Terry, J. K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L. S., Dieffendahl, C., Horsch, C., Perez-Vicente, R. et al. (2021), ‘PettingZoo: Gym for multi-agent reinforcement learning’, *Advances in Neural Information Processing Systems (NeurIPS)* **34**.

Terry, J. K., Black, B. and Jayakumar, M. (2021), 'SuperSuit: Simple microwrappers for RL environments'.

URL: <https://github.com/Farama-Foundation/SuperSuit>

Watkins, C. J. C. H. and Dayan, P. (1992), 'Q-learning', *Machine Learning* **8**(3–4), 279–292.

Williams, R. J. (1992), 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine Learning* **8**(3–4), 229–256.

Appendix A

Project Source Code Overview

This appendix provides a summary of the project’s source code structure. The full source code is available in the project repository.

Repository Structure

```
projet_rl/
|-- src/
|   |-- env_setup.py          # MuJoCo environment + reward shaping wrapper
|   |-- train.py              # PPO training pipeline
|   |-- evaluate.py           # Model evaluation with rendering
|   |-- soccer_match.py       # Real-time 2v2 match simulator with commentary
|   '-- utils/
|       |-- soccer_analytics.py # Coordinate tracking, possession & spatial hexbins
|       '-- analysis.py         # Statistical evaluation and trajectory graphing
|-- results/                  # Generated heatmap and evaluation curves
|-- Colab_Training_Notebook.ipynb # Google Colab vectorized training notebook
|-- requirements.txt           # Python dependencies
'-- rapport/                  # LaTeX report
```

Key Dependencies

Table A.1: Main Python dependencies used in this project.

Library	Purpose
stable-baselines3	PPO training framework
pettingzoo	Multi-agent environment API
shimmy	DmControl compatibility layer
supersuit	Environment vectorisation and parameter sharing
dm_control	MuJoCo physics engine
torch	Deep learning backend for PPO policy networks
tensorboard	Training curve monitoring
matplotlib	Publication-grade spatial heatmap plotting